LA POO SOUS PYTHON

MTA (tobmang1@gmail.com)

Chapitre III

Initiation à la POO sous Python

OBJECTIFS DU CHAPITRE

- Rappel des Concepts de bases de la POO en python
- Concepts Objets et Classes en python
- Quelques concepts avancés
- Écrire un programme complet et l’exécuter

•

Introduction

Introduction

- C’est quoi un programme : Ensemble d’instructions permettant à l’ordinateur d’effectuer des taches précises
- Logiciel : ensemble de programmes
- Trois catégories de logiciels :
  - Systèmes : utilitaires permettant un fonctionnement efficace de l’ordinateur (pilotes, compilateurs, éditeurs de textes, …)
  - De programmation : outils facilitant le travail des développeurs (EDI, debuggers, interpréteurs, …)
  - Applications : pour des taches spécifiques (traitement de texte, jeux, …)
- Evolution langages : **Machine → assembleur → évolué**

Paradigmes de programmation

- Programmation Impérative : le langage C
  - structurée, procédurale
- Programmation Logique : PROLOG
  - Extraction de la connaissance de faits et règles : Dire à l'ordinateur ce que sont les choses
- Programmation fonctionnelle : LISP, Haskel
  - Travail avec des fonctions
- Programmation Orientée Objet : Python*, Java, C#, C++*
  - Approche basée sur les objets et classes. Inclusion des notions d'encapsulation, héritage et polymorphisme.

Le python

- Python a été créé par Guido van Rossum en 1991 sous la version 1.0
- Présence en Industrie : Instagram(Django), Mozilla (Django), Paypal, eBay, Pinterest(Flask), Twilio (Flask), Amazon, Google, LinkedIn, NASA, Boing, Dopbox, …
- Contrairement à des langages comme C/Java (qui sont compilés), python est un langage interprété.
  - Dans un langage interprété les instructions sont transcrites directement en langage machine.
    - Simple et facilement portable, Moins rapide qu'un langage compilé, nécessite l'installation de python à chaque fois
  - Dans un langage compilé, un logiciel spécial se charge de la traduction en langage machine. Cette traduction est effectuée à chaque fois que le code est modifié
    - Difficilement portable (nécessite de changer le compilateur et quelques fois certaines instructions), Plus rapide qu'un langage interprété

Les Bases et variables

Les Bases et variables

- Python utilise l'indentation pour indiquer un block de code
  - Le non respect lève une erreur de syntaxe
  - Dans un même block il faut utiliser le même nombre d'espaces

- Les variables: **nom_variable = valeur**
  - Ce sont des conteneurs qui stockent des données
  - On ne déclare pas de variables sous python; la création se fait à la première affectation.
  - Nom : lettre, chiffres, underscore
    - Le nom commence par une lettre ou un underscore (et non un chiffre)
    - Le nom ne contient que des caractères alphanumériques et underscores
    - Langage sensible à la casse : mois # Mois.
    - Exemple de variable : mon_nom (**snake case**), monNom (**camel Case**), MonNom (**Pascal Case**).
    - Une variable ne peut porter le nom d'un mot_clé : **def, break, class, continue, …**
  - Assignations possibles: x,y=1,2 ; x=y=2 ; x,y,z = [1,2,3]

Les Bases: Les Opérateurs

Operator	Operation	Example	Evaluates to . . .
**	Exponent	2 ** 3	8
%	Modulus/remainder	22 % 8	6
//	Integer division/floored quotient	22 // 8	2
/	Division	22 / 8	2.75
*	Multiplication	3 * 5	15
-	Subtraction	5 - 2	3
+	Addition	2 + 2	4

- L'ordre de précédence est comme en Mathématique : de gauche à droite
  - **
  - * , / , // , et %
  - + et -

Les Bases: Les opérateurs et Types

- Opérateurs de comparaison: ==,!=, >, >= (possibilité de comparer les chaînes)
- Opérateurs Booléens : and, or, not
- Certains opérateurs facilitent la manipulation des variables : +=, *=, /=, -=.
- Sous python, la permutation se fait de la manière suivante : a,b = b,a
- Types basiques de données :
  - Nombres: Entiers: **int** , Décimaux: **float**[décimale sur 15chiffres],
  - **Boolean** (True, False),
  - **Strings** (single, double or triple cotes, 1 à n caractères),
  - **None** (représente l'absence de valeurs)

LES Exceptions

Gestion des exceptions

- **Exception:** Type levé par python en cas d'erreur ou de problème dans le code
- **Objectif**: récupérer les erreurs levées par python **try-except**
- Le mot clé **except** est utilisé pour lever les exceptions
- Toute exception hérite de la classe **Exception** (pour les exceptions **d'erreur**) qui elle hérite de **BaseException :**
  - Cette dernière inclut d'autres exceptions existantes qui ne soient pas des erreurs. Ex: **KeyboardInterupt** (ctrl+C) **.**
  - Pour gérer une majeure partie des exceptions en une ligne on peut tout simplement écrire **except Exception:**
- Les ASSERTIONS
  - Permettent de vérifier une condition avant de continuer
  - Ex: **assert** age > 18 -- AssertionErrors

LES STRUCTURES CONDITIONNELLES ET BOUCLES

Les structures conditionnelles et boucles

La décision

```
if cond :
        instruction
elif cond :
        instruction
else :
        instruction
```

La répétition

```
while cond :
        instructions
        condition arrêt
```

- Utile quand on veut itérer sous une certaine condition

Le parcours

```
For elt in sequence :
        instruction
```

- Permet d'exécuter un code plusieurs fois.
- utile lorsqu'on veut itérer sur une collection

LES STRUCTURES DE DONNÉES

Les chaines

- **Indexation**
  - En mémoire chaque caractère d’une chaine a un idex (location mémoire)
  - Le premier élément à une valeur 0, le second 1, …
  - Les crochets aident à indexer les éléments d’une chaine
  - **Ex :** mot = « bojour », print(mot[0]), print(mot[1]), print(mot[-1])
- **Slicing**
  - Utile lorsqu’on a besoin d’une sous-chaine : **var[start,stop,step]**
  - Ex : He dans s = Hello, print(s[0:2]) , print(s[0:5:2])
- Quelques méthodes : str.methode()
  - .title() : mettre en majuscule, .replace(c1,c2) : remplace c1 par c2, .find(str) : recherche la chaine, .strip(c) : élimine le caractère c à gauche et à droite,

LES LISTES

- Structure de données python dont on peut changer le contenu (muable) et de séquence ordonnée (son contenu est indexable).
- On accède aux éléments d'une liste par indexation
- Changement de valeur dans une liste
- L'affectation d'une liste à une autre variable se fait par **référence** ; modifier la seconde liste entraînera une modification de la première.
  - Une liste a une référence unique : **id(liste).**
- On utilise la fonction **copy** pour changer de référence et créer de nouvelles **listes id(l1) = id(l2)**
- **Quelques méthodes de liste : len(liste)** : taille, **le slicing** (start, stop, step), **liste.append(val)** : ajoute un élément, **liste.insert(index,val)**, **liste.pop(), liste.pop(0)** : retire et retourne un élément, **liste.remove(val)** : retire val de la liste

LES DICTIONNAIRES

- Collection non ordonnée de données sauvegardées sous forme de **key-value**
- Les dictionnaires sont utiles pour les grandes capacités de données, le mapping de donnée, le travail sous fichier csv, le transfert de données, …
- **Déclaration : dict = {} ou dic={key1:val1, key2:val2}.**
  - **Récupération d'un élément : var[key].**
  - **get** permet de récupérer une valeur à la seule différence qu'elle ne renvoie pas une erreur en cas d'absence de la clé mais le type **NONE.**
- Une liste peut être combinée à un Dictionnaire et vice-versa. Combinaison de list (accès par index) et dictionnaires (Accès par clé) :
- Dictionnaire dans un dictionnaire.

LES DICTIONNAIRES

- **Ajout d’une information : Fournir le nom du dictionnaire avec la clé-valeur à ajouter.**
  - La modification s’effectue de la même façon. Si la clé existe la valeur est modifiée
  - **les dictionnaires sont itérables comme des listes. on les parcours par clé et valeurs, clé ou valeurs.**
    - par clé avec **keys**()
    - Par valeurs avec **values**()
    - Par paires avec **items**()

LES TUPLES, SETS, FROZENSETS

- Les **tuples** sont des « listes » **immuables :** Une fois déclarés, on ne peut les modifier. Utile pour le stockage d'éléments qu'on ne veut pas modifier.
  - La déclaration d'un tuple peut se faire avec ou sans parenthèses.
- Les **sets** sont des collections assimilables au listes et dictionnaires, mais non ordonnées (non accessibles par index), qui ne peuvent contenir que des valeurs uniques (comme les clés d'un dictionnaire).
  - On les déclare soit avec le mot clé **set** + parenthèses + crochets, soit avec les accolades. **add(val)** et **remove(val)** sont des fonctions utiles avec les sets.
- Les **Frozensets** combinent les sets et les tuples : ils sont **immuables**, **non ordonnés** et uniques. Parfait pour des données sensibles comme des comptes bancaires (qui ne devraient pas être modifiés).
  - Déclaration : fs = frozenset([val]).

LA MODULARITÉ

Les Fonctions

- **Fonction** : groupe d'instructions qui s'exécutent lorsqu'on appelle son nom. **Ex** : print, type, input…
- Création de fonctions (Exemple table de multiplication)
  - **def nom_fonction (param1,…,paramN):**
    - **#Bloc d'instructions**
  - Les données sont passées sous forme de paramètres. **Ex** : param1=10.
  - Appels d'une fonction : fonct(), fonct(value), fonct(att=val).
  - En python, deux fonctions ne peuvent avoir le même nom ou la même signature. Il n'y a pas de surcharge en python.
  - L'instruction **Return** permet de renvoyer le résultat d'une fonction. Ex : return val

Les Fonctions

- Passage de paramètres
  - **Utilisation de *****args
    - Permet de passer un nombre variable d'arguments récupérés sous forme de tuple.
    - Utile quand on n'a pas d'idées précises du nombre d'arguments nécessaires
  - **Utilisation de ******kwargs
    - Permet de passé un nombre variable d'arguments identifiables par leurs clé
    - Le retour est un dictionnaire
- Le Scope des variables
  - Gère l'accès à nos variables. 3 types de scopes: global, fonction et classe.
  - **Variable globale :** accessible à un fichier ou à une application
  - **Variable de fonction :** déclarée dans une fonction et accessible seulement par elle.

La modularité

- **Module** : groupe de fonctions liées ayant un nom. Ex: math, random…
- **Les modules** : Ensemble de fonctions et de variables inter-liées.
  - Pour utiliser un module (fonctions et variables), il faut l'importer: import module
  - Appel de fonction ou variable avec : module.nom_fonction() / module.nom_variable
  - La fonction help permet d'avoir le contenu d'un module ou d'une fonction : help('module') ou help('module.fonction').
    - Tapez **Q** pour sortir de l'aide, **Espace** pour avancer et **Entrée** pour progresser d'une ligne.
  - Pour changer le nom d'un module on utilise le mot clé as :
    - import module as nouveau_nom
  - Il est possible de n'importer qu'un élément dans un module **ex :** une fonction :
    - from module import fonction.

La Modularité : code source dans un fichier

- But : plus de lisibilité et facilité de modification
- Ouvrir un éditeur standard (pyCharm, Notepad++,vs code, gedit, …)
- Inscrire le code dans le fichier et l'enregistrer sous l'extension .py
- Précisez l'encodage pour gérer les accents : Latin-1 (windows), Utf-8 (Lin, Mac) : # -*-coding:encodage -*
- Pour éviter une exécution rapide du programme (Sous windows), on peut le mettre en pause:
  - Importer en tout début le module os : import os
  - A la fin : os.system('pause')

La modularité

- **Exemple** : création de la fonction table de multiplication dans un module et un module de test

Fichier tables.py

```
#-*-encoding:Latin1 -*

#Module des Tables


def multiplication(n,maxi=10):

    """Affiche la table de multiplication de n jusqu'à
       maxi"""

    i = 0

    while i<maxi:

        i+=1

        print(i,"*",maxi," = ",i*maxi)
```

Fichier test.py

```
#-*-encoding:Latin-1 -*
#Teste mes tables

import os
from tables import multiplication

multiplication(2,20)

os.system("pause")
```

La Modularité : code source dans un fichier

- **Cas d'une exécution dans le même fichier**

```
#-*-encoding:Latin1 -*
#Module des Tables
import os

def multiplication(n, maxi=10):
    """Affiche la table de multiplication de n jusqu'à maxi"""
    i = 0
    while i<maxi:
        i+=1
        print(i,"*",maxi," = ",i*maxi)

#Exécution dans le même fichier
if __name__ == "__main__":  # utile quand on veut importer un
                            #module sans l'exécuter.
    multiplication(3,40)
    os.system("pause")
```

La Modularité : Les packages

- Les modules permettent de regrouper plusieurs fonctions
- Les packages regroupent plusieurs modules.
- Importer un package : *import nom_package.module*
- Importer un module dans un package : from *package* import *module*
- Création d'un package
  - Créer un répertoire portant le nom du package
  - Dans le répertoire créer un fichier __init__.py (Ce fichier peut être vide)
    - Un package peut contenir des fichiers .py
    - Ou d'autres packages (créés de la même façon)
- Exemple : créer un package et y introduire un fichier multiplier.py.
  - Dedans introduire la fonction table
  - Testez la fonction dans un autre fichier.

NOTION DE CLASSES

Les Notions de Classes: **Introduction**

- La Classe est un modèle suivant lequel on peut créer des objets
- **Convention:** sous python on utilise la convention de nommage **Camel Case;**
  - **Camel Case** n'utilise pas de **Underscore** pour séparer les mots. Le principe est de mettre les premières lettres de chacun des mots en majuscule (**LaClasse**).
- Le mot clé **class** permet de définir une classe: **class LaClasse**
  - Ex: la classe Personne caractérisée par un nom complet, sexe, résidence
- Toute classe a un **constructeur**. Il est utilisé pour créer un **objet et ses attributs**.
  - On utilise la fonction **init** est une méthode spéciale utilisée pour définir un constructeur: **def __init__(self)**
    - **Self** représente l'objet en cours de création

Les Notions de Classes

- On instancie une classe en l'appelant: **objet = LaClasse()**, **objet.attribut** permet d'avoir la valeur de l'attribut de l'objet.
- A l'appel d'un constructeur, l'objet **self** est renvoyé
- Dans le constructeur, on peut initialiser des valeurs avec (**self.attribut = val**)
- On peut modifier un attribut en lui affectant directement une valeur: **objet.attribut=val2.**
- Un constructeur peut prendre plusieurs paramètres: **def __init__(self, nom, sexe):**
- Sous python, les méthodes sont des **actions** agissant sur l'objet
  - **def methode(self,parametre)**
- **LES ATTRIBUTS DE CLASSE**
  - Ce sont des attributs qui ne dépendent pas de l'objet
  - Ils sont propres aux classes et sont définis directement dans le corps de la classe.
  - On les appel en saisissant **Classe.attrib ,** et dans une méthode **self.attrib**

Les Notions de Classes

- **LES MÉTHODES STATIQUES**
  - Méthodes propres à la classe et ne dépendant pas de l'objet
  - Elles ne sont pas accessibles à l'objet et sont définies sans le mot clé **self**
- **Introspection** = Connaître le contenu d'un objet
  - **dir(objet)**: retourne le contenu d'un objet: attribut, méthode… **ex : dir(str), import math, dir(math)**
  - **__dict__**: **attribut spécial** contenu dans tout objet d'une classe créée. C'est un Dictionnaire {nom_attribut:val_attribut}. **ex : créer une classe, l'instancier et tester. obj.__dict__**
  - **NB**: Il est possible de modifier les valeurs du dictionnaire: **obj.__dict__[attrib] = val**

Les Notions de Classes: **Méthodes spéciales**

- **Compréhensibles par python: __init__()** permet la création d'un objet
- Les méthodes spéciales sont celles entourées par des doubles **underscores.** Elles permettent d'éviter que python prenne un comportement par défaut définit dans la classe **object.**
  - __del__(self) est appelée à la destruction de l'objet. En utilisant le mot clé **del(obj)**
  - __repr__(self) utilisé pour faciliter la représentation d'un objet lorsqu'on saisit son **nom** ou **repr(obj)**
  - __str__(self) pour l'affichage dans un print. **Repr** est utilisé par défaut si elle n'est pas définit.
  - __getattr__ utilisée pour l'accès à un attribut inexistant avec **obj.attribut**.
  - __setattr__ appelée par python pour une modification de valeur: **obj.att = val**
  - __delattr__ est appelé lors de la suppression d'un attribut: **del obj.att**

Les Notions de Classes: **l'héritage**

- Si **F** hérite de **G** alors les objets de **F** auront accès à toutes les méthodes et attributs de **G**
- L'héritière (La classe) est capable de redéfinir les méthodes de la classe mère et, de définir d'autres méthodes et attributs qui lui sont propre
- L'héritage peut être simple:
  - **Simple: A** hérite de **B** : **class A(B):**
  - **Multiple**: **A** hérite de **B** et **C**. **class A(B,C)** : on utilise la classe au lieu de super.
- **Super** est utilisé lorsqu'on doit instancier des attributs dans la classe mère à l'instanciation de la fille.
- L'héritage permet la customisation ou redéfinition des méthodes

Petit TP

- **Simuler des matching Homme-Femme.**
- Créer un package (match_tp) **sans oublier le fichier __init__.py**

```
Début de la phase d'enregistrement
Donner le nom,cni,age de la personne à créer
Taper 1 pour un homme et 2 dans le cas contraire 1
Donner le nom de l'homme: Manga
Donner le numéro de cni: 123
Donner l'age de l'homme: 34
voulez-vous continuer? 0 pour'oui' et 9 pour 'non' 0
Donner le nom,cni,age de la personne à créer
Taper 1 pour un homme et 2 dans le cas contraire 2
Donner le nom de la femme: erd
Donner le numéro de cni: 23
Donner l'age de la femme: 21
voulez-vous continuer? 0 pour'oui' et 9 pour 'non' 9
Fin de l'enregistrement
Combien de couples voulez-vous? 1
Il est probable que Manga soit en couple avec erd
Press any key to continue . . .
```

Concepts Avancés

COMPRÉHENSION DE LISTES

- Initialement pour introduire des données dans une lite on procède comme suite :
  - On la crée vide
  - On crée les données par itération
  - On append les données dans la liste.
- La compréhension des listes permet d'effectuer ces taches en une instruction.
  - ***Resultat = [transformation iteration filtre]***
  - ***Ex :*** *liste = [elt_a_ajouter **for** el **in** list]* ou *liste = [elt_a_ajouter **for** el **in** list **if** condition]*
    - *liste = [elt_a_ajouter **if** condition **else** elt_a_ajouter **for** el **in** list]*

Fonctions LAMBDA

- Aussi appelées fonctions **anonymes**, elles sont écrites sur une ligne.
- Elles aident à réduire le nombre de lignes de code dans le programme
  - Utile pour la lecture du code mais pas possible pour des fonctions complexes
- **Syntaxe : lambda arguments : expression**
  - **lambda arguments : valeur_de_retour if condition else valeur_de_retour**
- Par défaut, les fonctions lambda retournent le résultat.
  - Plus besoin du **return**.
- On met la fonction lambda entre parenthèses pour l'exécuter directement.
  - Si non, le résultat peut être sauvegardé dans une variable.
- Les conditions dans le Lambda se font nécessairement avec un **if** et un **else**

Les Décorateurs

- **Décorateur :** Moyen Simple de Modifier le comportement par défaut d'une fonction ou d'une classe.
  - Il prend en paramètre et retourne une fonction.
  - Il offre d'autres capacités aux fonctions sans avoir à modifier leur code.
  - On décore une fonction en ajoutant @**decorateur** au dessus de la fonction
  - Il est utile lorsque vous voulez exécuter des fonctionnalités avant ou après l'exécution d'une fonction.
  - l'exécution d'une fonction décorée se fait initialement par l'appel du décorateur qui décide de la suite et des conditions.
  - **EX d'utilisation:** Si l'on veut limiter l'accès à une fonction aux utilisateurs connectés, au lieu d'introduire la même condition dans chaque fonction, on peut mettre le code dans un décorateur et l'appliquer à ces fonctions.

Les Décorateurs

- Décorer une fonction revient à déclarer que son comportement normal peut être modifié par une autre :
  - La modification se fait à la définition de la fonction pas à son appel : c'est le décorateur qui décide du résultat final
  - Cela se fait par l'ajout du nom du décorateur avant la définition de la fonction :
  - @decorateur sur une fonction équivaut à f2 = decorateur(fonction)
    - **Ex** : de la création d'une classe singleton
  - Un décorateur avec les paramètres vas donner f2 = decorateur(parametre)(fonction)
  - L'action des paramètres sur le décorateur permet de retourner un autre décorateur, qui lui retourne une fonction.

Les Expressions régulières

- Les expressions régulières ou **regex** constituent la description d'un patron dans un texte.
  - Ce sont un moyen d'isoler de rechercher ou de remplacer des expressions dans une chaîne
- \d : un digit – un caractère compris entre 0 et 9. ex : \d\d\d-\d\d\d-\d\d\d
- L'ajout d'un nombre entre accolade après un pattern match le pattern nb fois : \d{3}
- Les fonctions **regex** sous python sont définies dans le module **re** (import re)

L'utilisation des parenthèses permet de définir des groupes. Ainsi il est possible de signifier l'objet matché à afficher avec group(numéro) : (\d\d\d)-(\d\d\d-\d\d\d\d)

- Utiliser un pipe pour matcher plusieurs expressions :

Les Expressions régulières

- Si les 2 apparaissent, la première occurrence est retournée : l'utilisation de la méthode **findall**() permet de retourner toutes les occurrences.
- On peut utiliser un préfixe si les termes en partagent un. Ex : Bat pour 'Batman'
- L'appel de mo.group() retourne tout le texte matché 'Batmobile', Tandis que mo.group(1) retourne la partie du texte matchée dans la parenthèse 'mobile'

- Controler le nombre d'occurrences d'un caractère
  - ? (traitement optionnel, 0 ou 1), ex : 'ab', 'abc'
  - * (0 ou plusieurs), ex : abc* = 'ab', 'abc', 'abcc', 'abccc', …
  - +(1 ou plusieurs), ex : abc+ = 'abc', 'abcc', abccc', …
  - {} (numbre specifique), ex : E{4} : 4 fois la lettre E. E{2,4} : de 2 à 4 fois.

Les Expressions régulières

- Utiliser les classes de caractères
  - [a,b,c,d] : une des lettres parmi a,b,c et d.
  - [A-Z] : une lettre en majuscule
  - [A-Za-z0-9] : une lettre, majuscule ou minuscule, ou un chiffre
  - [A-Z]{5} : 5 lettres majuscules qui se suivent.
- Les groupes : Appliquer le contrôle d’occurrence à plusieurs caractères
  - (cha){2,5} : expression vérifiée pour les chaînes contenant la séquence ‘cha’ répétée de 2 à 5 fois.

Conclusion

- Rappel des Concepts de bases de la POO en python
- Concepts Objets et Classes en python
- Quelques concepts avancés : décorateurs, Regex, compréhension de listes, lambda
- Écriture des programmes python